

AIMS | African Institute for Mathematical Sciences CAMEROON

Multi-Category Quality Assessment of Safou (Dacryodes edulis) Using CNN Models

Kochiake Kanne Tamibe (kochiake.kanne@aims-cameroon.org)
African Institute for Mathematical Sciences (AIMS)
Cameroon

Supervised by: Dr Arnaud Nguembang Fadja
Department of Engineering, University of Ferrara, Italy

28 May 2025
Submitted in Partial Fulfillment of a Structured Masters Degree at AIMS-Cameroon

Abstract

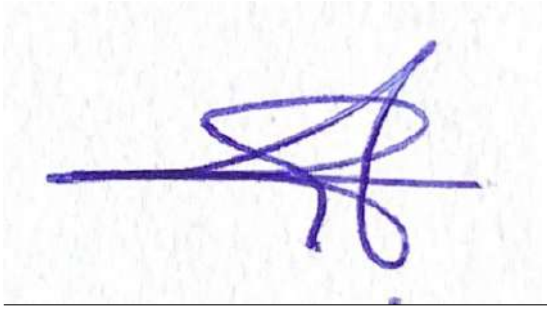
Ensuring the quality of agricultural products, minimizing post-harvest losses, meeting consumer preferences, and maximizing market profitability are central objectives of automated agricultural sorting systems. In the context of Dacryodes edulis (commonly known as Safou), sorting is still performed manually through visual inspection, a method that is inherently inefficient, inconsistent, and labor-intensive. To address these limitations, this thesis explores the application of Deep Learning, particularly Convolutional Neural Networks (CNNs), for the multi-class classification of Safou fruits according to various quality grades: bruised, cracked, spotted, rotten, unripe, and unaffected.

A dataset comprising 4,507 annotated images was utilized, and identical preprocessing techniques were applied across all models. Three state-of-the-art CNN architectures DenseNet-121, ResNet-50, and ReXNet-150 were implemented and evaluated. The ResNet-50 model outperformed the others, achieving an accuracy of 91.07%, followed by ReXNet-150 with 90%, and DenseNet-121 with 88.4%. These results highlight the effectiveness of ResNet-50 in capturing fine-grained visual differences across quality categories, thereby demonstrating its suitability for automated fruit classification in post-harvest processing systems.

Key words: Post-Harvest Sorting, African Plum, Dacryodes edulis, Fruit Quality Classification, Convolutional Neural Networks, Agricultural Automation.

Declaration

I, the undersigned, hereby declare that the work contained in this essay is my original work, and that any work done by others or by myself previously has been acknowledged and referenced accordingly.



Kochiake Kanne Tamibe, 28 May 2025.

Contents

Abstract i

1 Introduction 1

1.1 Background . . . 1

1.2 Problem Statement . . . 2

1.3 Research Objective and Contributions . . . 3

1.4 Thesis Organisation . . . 3

2 Literature Review And Machine Learning Architectures 4

2.1 Introduction . . . 4

2.2 Background of Study . . . 4

2.3 Previous Studies on fruit quality detection in Agriculture . . . 5

2.4 Challenges in image classification Agriculture . . . 6

3 Convolutional Neural Networks(CNN) 7

3.1 Introduction to CNN . . . 7

3.2 RestNet50 . . . 11

3.3 DenseNet121 . . . 13

3.4 RexNet-150 . . . 14

4 Implementation 16

4.1 Data annotation and labeling . . . 16

4.2 Data processing . . . 17

4.3 Model training and validation . . . 19

5 Results and discussion 20

5.1 DenseNet-121 . . . 20

5.2 RexNet-150 . . . 22

5.3 Resnet-50 . . . 23

Acknowledgements 28

References 30

1. Introduction

New technologies like computer vision and artificial intelligence (AI) are becoming increasingly relevant in areas such as health care, agriculture and many others. This expansion is more evident in the developed countries, where technology has evolved considerably. Because of the availability of massive data and machine learning algorithm improvement, computer vision and AI became powerful and indispensable for object classification, detection and segmentation tasks. In agriculture, this advancement is marked by the automation of crop monitoring, diagnosis of plant's diseases and the assessment of the quality of agricultural products. All these applications help to maintain productivity, improve work efficiency, less time consuming, reduce labor costs with the aim of ensuring delivery of healthy and quality products to the market.

Dacryode Edulis (Safou) is a fruit which is mostly cultivated in Center and West Africa [1], and it is the focus of our research. This fruit is of vital socio-economic importance in several regions of Cameroon [1]. However, its market value is affected by quality defects like degraded textures. Our objective was to implement DenseNet-121, ResNet-50, and RexNet-150 deep learning models based on convolutional neural networks for the classification of African plums according to their quality states. These models demonstrated strong effectiveness in accurately identifying various defect categories, including spotted, unripe, cracked, bruised, rotten, as well as the unaffected (good quality) state.

1.1 Background

In Cameroon, Safou is of major importance for South, Center, South-west communities where it is mostly grown in association with coffee and cocoa in forests [1]. Figure 1.1 shows the two parts of a safou, namely the flesh and the seed. It is characterized by its color, which ranges from light pink (unripe) to maroon blue (ripe) as shown in figure 1.2.



Figure 1.1: Component of Safou [2]



Figure 1.2: Safou [2]

It is used for nutritional, medicinal and other important purposes [1]. Its fruits are rich in nutrients such as lipids, proteins and carbohydrates; its leaves and barks are used in traditional African medicine to treat anemia, skin infections, wounds, fever, diarrhea among other illnesses. Its trunk and branches are

used as fuel, and are sometimes transformed into utensils or craft tools.

Besides being nutritious, it is one of the best-selling wild fruits in the country. Its national commercial value was estimated at 7.5 million US dollars in 1997 for 11,000 tons according to [1, 3]. Given that its characteristics define its market value, it is therefore necessary to assess its quality. To do that, after harvesting, the workers sort fruits by hand, separating the bad plums from the good ones to prevent them from being affected by the bad ones. This sorting method is based on traditional visual quality control. This task is tedious, inefficient and time-consuming especially on a large scale. It is therefore necessary to integrate technologies such as computer vision and AI to automate this process and make it faster, more efficient and more coherent, with the aim of improving yields, reducing post-harvest losses, meeting specific consumer needs and quality standards for export or industrial processing.

1.2 Problem Statement

Agriculture has always been essential for both survival and socio-economic development throughout the world. The more it is practiced, the more it enables us to maintain the continuous supply and production that is fundamental necessity. However, the set of chain supply is facing to increasing challenges as the loss of crops due to transportation or bad sorting of crops. Sorting crops has always been an important process that allows to ensure good quality of crops in the market and reducing post-harvest losses. From the dawn of time to today, this process remains a necessity for both consumers, transformation industries and farmers. Nevertheless, this method remains manual i.e by visual inspection by humans and based on some criteria such as the color, texture and size. What's more, this method has proved to be imprecise, inefficient and difficult.

Over the last decade, several deep learning models have been implemented to automate this task and make it efficient, especially for fruit. Concerning Safou, until nowaday this sorting remains binary (bad or good) which is not good for either consumers or retailers. Hence the need to assess fruit quality and condition. This qualitative classification of defects helps to reduce food waste, optimize economic value, improve customer satisfaction and ensure that defects are traceable. Hence the aim of our research, the implementation of a qualitative classification model for African plums based on the CNN architecture as shown in figure 1.3 where ResNet-50, RexNet-150 and DenseNet-121 are used.

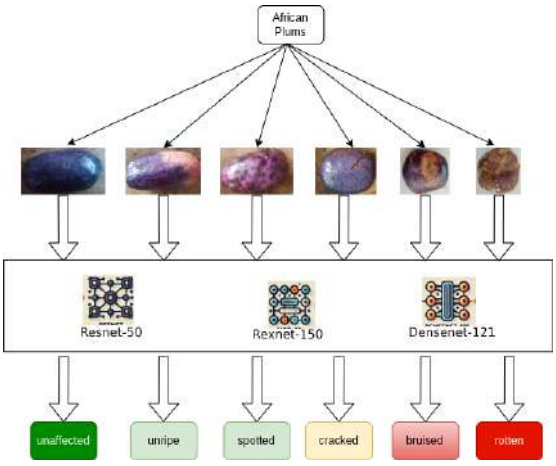


Figure 1.3: Classification of plums

1.3 Research Objective and Contributions

1.3.1 Objective. The aim of our research is to build models which are able to classify well African plums according to whether they are bruised, cracked, rotten, spotted, unripe or unaffected based on DenseNet, Resnet and RexNet architectures.

1.3.2 Contribution.

- Some images were initially misclassified, but through visual verification, we were able to identify and subsequently reclassify them based on their characteristics.
- We developed and trained DenseNet-121, ResNet-50, and RexNet-150 convolutional neural network architectures to automatically extract discriminative features associated with different quality states of plums and perform accurate classification based on these visual characteristics.
- Evaluation and optimization: Although the models initially exhibited good performance, the results were not entirely satisfactory. To enhance their accuracy, we optimized them by adjusting key hyperparameters and applying techniques such as fine-tuning and replacing the final classification layer. These modifications led to notable improvements in model performance

1.4 Thesis Organisation

This thesis is structured into six (6) chapters, each highlighting essential points of our research. Chapter 1 is the introduction that aims to present African plums and quality-related challenges (background), limitations of classic methods applied for ensuring good quality (problem statement) and finally the objective and contributions of this research.

In Chapter 2, different methods used and their limitations in previous researches on quality detection of fruits in agriculture are highlighted, and underlined how our contribution brings solution to these limits.

Chapter 3 presents the architecture of CNN and that of the models used to implement solutions based on DenseNet-121, ResNet-50 and RexNet-150.

Chapter 4 presents the methodology, consisting of data annotation and pre-processing, model training and evaluation.

Chapter 5 shows results obtained by each of models, followed by their interpretation and discussion.

Finally, we have conclusion and futur work which briefly recalls the research objective, methodology, results obtained, research limitations and finally future work.

2. Literature Review And Machine Learning Architectures

2.1 Introduction

Recent advancements in Deep Learning, particularly Convolutional Neural Networks (CNNs) have demonstrated significant potential in the domain of agricultural automation, especially for the classification and detection of fruits. This chapter presents an overview of contemporary research efforts leveraging CNN-based models for fruit quality assessment and object detection. While a broad spectrum of studies has explored fruit classification tasks, our focus will narrow toward the detection and quality evaluation of Dacryodes edulis, commonly known as the African plum.
Accurate classification of crops plays a crucial role in modern agriculture, ensuring the delivery of high-quality products to both local and international markets. Traditionally, this process relies heavily on manual inspection, where human evaluators assess fruit quality based on visual clues. However, such manual approaches are inherently slow, labor-intensive, and prone to inconsistencies due to subjective judgment.
To address these challenges, deep learning techniques particularly CNNs have emerged as powerful tools capable of automating the visual inspection process. By learning discriminative features directly from images, these models offer a scalable, objective, and efficient alternative to manual classification, paving the way for smarter and more productive agricultural practices.

The objective of this section is to bridge the gap between previous research and the current study by providing a comprehensive understanding of developments in fruit detection and classification. Through a comparative review of both conventional and deep learning-based approaches, we emphasize the improvements achieved in terms of accuracy, processing speed, and operational efficiency. This review also reveals existing gaps, notably the limited focus on African plums (Dacryodes edulis) and the persistent challenges introduced by varying environmental conditions. Building on these insights, our study advocates for the use of advanced deep learning architectures such as DenseNet, ResNet, and RexNet. Unlike prior works that primarily address binary classification, our approach targets multi-class classification, distinguishing plums based on specific quality categories, including unripe, unaffected, cracked, bruised, stained, and rotten.

2.2 Background of Study

It is well established that sorting constitutes a critical step in the post-harvest management process, significantly influencing both the quality and commercial value of agricultural produce. Market value is largely determined by factors such as ripeness, skin condition, visual attributes (including size, shape, and color), texture, and taste. As such, accurate detection and quality evaluation are essential before products reach consumers. Traditionally, this sorting process has relied on manual inspection, which is labor-intensive, time-consuming, and often inconsistent. To overcome these limitations, emerging technologies like computer vision and artificial intelligence have shown promising potential. In particular, Convolutional Neural Network (CNN) architectures such as ResNet, RexNet, DenseNet, Fast R-CNN, and Mask R-CNN have demonstrated strong capabilities in extracting visual features related to

texture, shape, and color. These models are increasingly being adopted in a wide range of agricultural applications, including disease detection, fruit quality assessment, and species classification.

2.3 Previous Studies on fruit quality detection in Agriculture

The detection of fruit quality remains a crucial task within the agricultural sector. In recent years, there has been growing interest among researchers in addressing this challenge, particularly through the application of Convolutional Neural Networks (CNNs) for fruit classification and detection. To achieve this, various approaches have been proposed, ranging from traditional image processing techniques to more advanced deep learning-based methods.

2.3.1 Traditional method. According to [4] a model for detecting apple maturity based on simple neural network including applied techniques are segmentation by threshold, color analysis and transformation of 3D images into 2D has been developed. This model performs well with 94% of accuracy, but it presents technology limitations where the extraction of features is done manually and the sensibility to lighting variations.

2.3.2 Method based on Deep learning. Some models have been developed for fruits detection using deep learning such as: apple, bananas, avocados, orange, and many other. Most of these models are based on architecture like, Faster RCNN, Mask RCNN and YOLO.

- In 2016, Sa, Inkyu, et al.[5] developed detection system for sweet pepper using RGB (Red-Green-Blue) images and NIR (Near-Infrared). The aim was to make this task effective in time and precision based on Faster RCNN with VGG16 as basic architecture. It is able to detect this fruit at 83.8% , then it has been generalized for seven others fruits (bananas, apples, oranges, mangoes, avocados, strawberries) with a performance of over 80% of f1 score for each fruit. Even though this model performs well on several species of fruit it has some limits such as variation of the size of fruits that makes the detection difficult, computational capacities are limited and annotation is still manual.
- In 2020, according to [6] Mask RCNN has been trained for apples detection to improve visual perception of robots during harvesting. This model has as basic architecture the combination of ResNet and DenseNet to allow best extraction of feature and it shows best performance with 97.31% of accuracy. Even though this performance is good, the model is limited by the disability to generalize well with other fruits, the computational complexity and the small data size.
- Goyal, K., Kumar, P., & Verma, K. (2023) [7] developed another model for identifiaction and quality detection of four (04) fruits (banana, apple, orange and tomato). Here, these fruits are classified depending on whether they are ripe, more ripe, unripe and defect compared with the studies cited above, which focused on detection by fruit type only. Fruit identification achieved a recall of 91% and a mAP of 92.80%. As for quality detection, the best performance was on apples with 99.84% F1 score and a mAP of 99.7%, while bananas showed the lowest performance at 86.9% mAP. Despite the best performance, the model is limited by the sensibility to bad quality of images with noise on the background, similarity between classes (ripe, more ripe), the complexity to make a difference between fruits and background. Also, it is limited to four (04) fruits that need to generalize to other fruits.

As far as African plums are concerned, few studies are available. Among them, Sain Rigobert et al.[8]

used several models (three different versions of YOLO, Fast R-CNN, Mask R-CNN, VGG-16 and four others models based on CNN) and YOLOv8 is the one that obtained a best performance with 93.6% of mAP, 87% of precision on a set of 2,800 images, obtaining a F1 score of 85.1%. However it is limited on just two classes (Bad and Good), also by the small data size, the sensitivity to complex real-life conditions such as light, angles, background and lack of precision in fruit quality.

All these methods are limited to few classes not representative of the different states of a fruit that we can have and allow sale differentiations and segmentation of fruits, reduction of wasting by recycling slightly damaged and unripe fruit for other useful purposes, offering products adapted to customer needs. These limitations justify this research on Multiclassification of African plum into 06 classes which are unripe, bruised, unaffected, cracked, spotted and rotten using DenseNet-121, RexNet-150 and ResNet50.

2.4 Challenges in image classification Agriculture

Image classification in the agricultural domain is a vital task, however it presents several challenges. These include the diversity of crops varying in size, color, and shape the poor quality of available images, and the limited quantity of labeled data. To ensure accurate classification, images must be captured under conditions that realistically represent the natural environment, adhering to key criteria such as consistent viewing angles, coherent brightness levels, diverse visual representations of crops, and precise image annotation. Failure to meet these standards often results in misclassification and unreliable model outcomes. Furthermore, challenges such as deceptive textures, where the object cannot be easily distinguished from the background, must be addressed. It is equally important to maintain class balance and ensure the dataset is of sufficient size to prevent bias and overfitting. Addressing these factors is essential to overcoming the inherent difficulties of agricultural image classification and developing robust and effective deep learning models.

3. Convolutional Neural Networks(CNN)

3.1 Introduction to CNN

Artificial intelligence is the subject of several researches nowaday, especially in computer vision whose aim to allow computer to understand images (or video, signal audio) and interpret them as human would do it [9]. Moreover computer vision tasks are mostly based on neural networks, especially convolutional neural network (CNN) [10]. CNNs are architectures deep learning especially builded for images processing tasks such as features extraction that is the principal task, classification, detection and segmentation of images. In our time, CNN impose themselves and excel in resolution of several complex tasks in many areas such as agricultural, medicine, safety and others areas [9]. CNNs are powerful for analyze images contrarily to simple neural network by the fact that they preserve spacial relationships between pixels.

3.1.1 Architecture. The architecture of a CNN typically consists of four (04) main types of layers excluding the input and output layers each serving a distinct function that contributes to the model's overall performance. These core layers include the convolutional layer, the activation function layer (commonly using ReLU), the pooling layer, and the fully connected layer. Together, they enable the network to learn hierarchical feature representations from input images, facilitating effective classification and detection tasks.

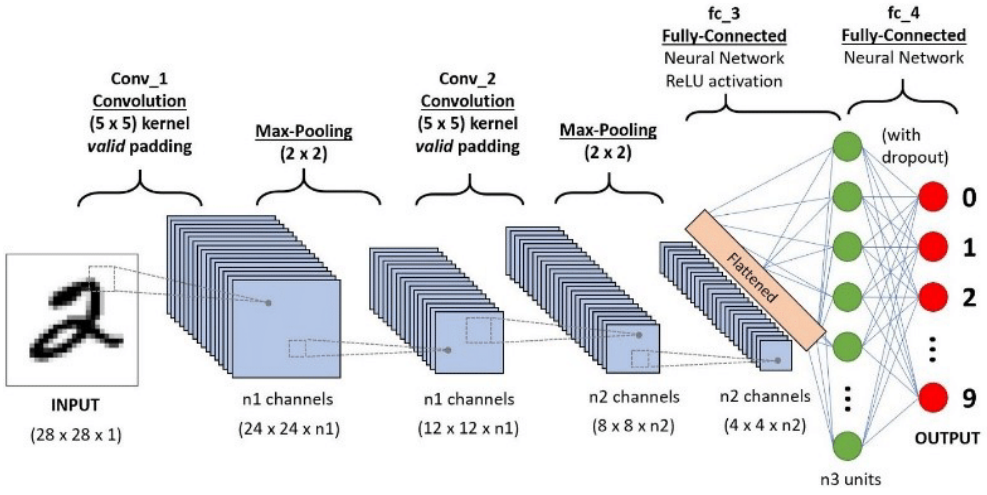


Figure 3.1: Architecture of Convolutional Neurone Network (CNN) [11]

- Convolution layer: This layer allows feature extraction automatically of image by sliding several filters over the input image at a defined pitch to enable the model to see and recognize various aspects without loss spacial relation between pixels. That consists of matrix product of the filter and pixels to produce the characteristic map. This characteristic map contains different motifs of image like textures, shapes, edges and more [12] and it allows to just save important structures. In addition, in this layer the weights of a single filter are shared throughout the image to ensure translational invariance, and non-linearity is then applied (activation function) [13].

The operation of convolution is defined mathematical by:

$$F(x,y) = \sum_{i=0}^{m-1} \sum_{j=0}^{m-1} f\,(x+i,\ y+j) \cdot h(i,j) \tag{3.1.1}$$

Where $F(x,y)$ is the pixels of the output image, $f(x,y)$ pixels of input image, $h(i,j)$ is the kernel of $m \times m$ (it's a matrix of weights learned during training, the dimensions of which are defined beforehand). As example we have the following figure illustrating the mathematical operation of convolution using 3x3 kernel and stride = 1:

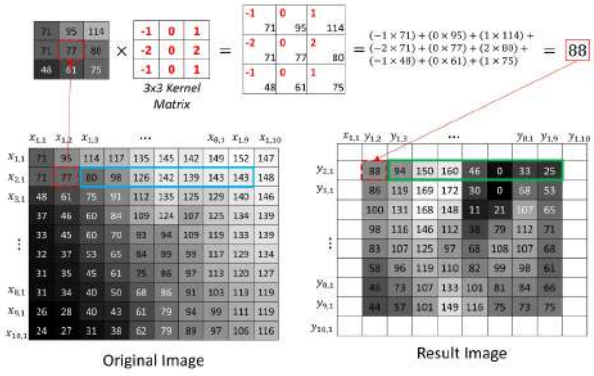


Figure 3.2: Convolution of 3x3 kernel matrix to a 10x10 image matrix [14]

Convolution is applied to an image of size 10x10 with a kernel of size 3x3 and the output is a matrix of size 8x8.

- Pooling layer: This layer allows reducing of spacial dimensionality of feature maps while maintaining the most important information and reducing the number of parameters and computational complexity [13]. There are many types of pooling but max and average are the most using [15]:

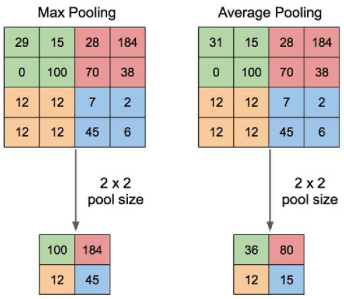


Figure 3.3: Max and Average pooling [15]

  - Max pooling: A window or kernel slides on feature maps, and only the maximum value is saved for each window. This helps to reduce the size of output, translational invariance and highlighting dominant features such as edges, textures and more.
  - Average pooling: The principle is the same as for max pooling except the fact that the average of values in each sliding window is calculated and retained. It allows to keep global information and reducing of noise.

- Fully-connected layer [13]: Fully-connected layers are primarily designed to perform prediction task in CNN such as classification or regression based on characteristics extracted by convolutional and pooling layers that are first transformed into a 1D vector. In this layer each neuron is connected to all neurons from the preview layer. This layer is generally at the end of network follow a sequence of convolutional and pooling layers. Full-connected layer allows global making-decision by taking into account all features extracted from the image.

- Non-linear activation functions [16]: Activation function is defined as a mathematic function applied to the output of a neuron in the network transforming the weighted sum of inputs into a non-linear output value and decides whether the neuron should be active or not.

  However, CNN without activation function is not able to learn complex patterns. Therefore, CNN works with 03 important components that allow to get the output as is defined by the following formula:

$$\text{Output} = f\left(\sum_i (w_i \cdot x_i) + b\right) \text{ [16]} \tag{3.1.2}$$

  where w_i is weight, x_i is the input, b is the bias and f the activation function.

  There are several types of non-linear activation function in deep learning based on their roles:

  a- Sigmoid function [17]: This function is an excellent choice for modifying probabilities, especially in classification by the fact that it transforms inputs into values between 0 and 1. However sigmoid function raises the problem of vanishing gradient for big values of inputs and it defined by:

$$\sigma(x) = \frac{1}{1 + e^{-x}} \tag{3.1.3}$$

  where :

  * x is the input of neuron (weighted sum of inputs + bias)
  * output $\sigma(x)$ is always between 0 and 1.

  b- ReLu (Rectified Linear UNit) function [17]:

$$\text{ReLU}(x) = \max(0, x) = \begin{cases} x & \text{if } x \geq 0 \\ 0 & \text{if } x < 0 \end{cases} \tag{3.1.4}$$

  The output is always positive and belong $[0; \infty[$. It is less costly to calculate than others and accelerate training by reducing the number of active neurons at once. But raises the problem of vanishing gradient especially when inputs become so negatives.

  c- Tanh activation function [18]:
  This function is an optimized version of Sigmoid with balanced weight updates thanks to the fact that outputs are around zero and allows accelerated convergence during training. However, TanH raises also the problem of vanishing gradient and higher calculation costs than ReLu. This function is defined by:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} = 2\sigma(2x) - 1 \tag{3.1.5}$$

  where $\sigma(x)$ is the sigmoid function and x the input.

d- Softmax:
Softmax is only used for classification task in CNN. It generally uses toward the end of the architecture after the last fully-connected layer to transform components of the vector to probabilities (the sum of these probabilities should be 1) where each value is a probability to be belong a given class. This function is defined by:

$$\mathrm{SF}(x_i) = \frac{e^{x_i}}{\sum_{j=1}^{N} e^{x_j}} \tag{3.1.6}$$

3.1.2 Loss function. CNNs are powerful tools for feature extractions whose performance is based on the ability to minimize the error between true and predicted data. To achieve this, it is important to note that there are loss functions whose choice depends only on the type of task to be performed. For classification tas, we distinguish between Cross-entropy for multiclassification and binary cross-entropy for binary classification:

- Cross-entropy for multiclassification. It is used in multiclassification tasks where we have at least three (03) classes and can be expressed mathematically as:

$$\text{Cross-entropy}(\mathbf{y}, \hat{\mathbf{y}}) = -\sum_{i=1}^{C} y_i \cdot \log(\hat{y}_i) \tag{3.1.7}$$

  Where C represents the number of classes, y_i the true class of the given instance i, and $\hat{y}_i$ the predicted probability.

- binary cross-entropy. It is used for binary task where we have 2 classes and expressed mathematically by:

$$\text{Binary cross-entropy}(y, \hat{y}) = -\left[y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y})\right] \tag{3.1.8}$$

  Where y is the true classe and $\hat{y}$ the predicted probability of this classe.

3.1.3 Evaluation metrics for CNN models. There are several metric's evaluation of CNN models which allow to ensure reliability and relevance of these models. Namely, for classification tasks we have such as : accuracy, confusion matrix, precision, recall et f1 acore.

- Confusion matrix. This metric provides a detailed classification report, comparing the predictions with the true labels as shown in the table 3.1 for the binary classification case:

Table 3.1: Confusion matrix for binary classification

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

TP is the number of correctly predicted positive instances; TN the number of correctly predicted negative instances; FP is the number of negative instances wrongly predicted as positive and FN the unmber of positive instances wrongly predicted as negative.

- Accuracy. This metric quantifies the ratio of overall correctness of predictions. It is determined from confusion matrix and defined by the following formula:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}} \tag{3.1.9}$$

- Precision. It is also computed from the confusion matrix. This metric represents the ratio of true positive predictions among all positive predictions made by the model. IT is defined by:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \tag{3.1.10}$$

- Recall. It measures the model's ability to correctly predict all instances of the positive class:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \tag{3.1.11}$$

- F1 score. It is defined by the harmonic mean of precision and recall :

$$\text{F1 score} = 2 \times \frac{\text{Precision x Recall}}{\text{Precision} + \text{Recall}} \tag{3.1.12}$$

All these metrics are used in appropriate situations, accuracy has a meaning for balanced data because it would be biased by the majority class. On the other hand, in the case of unbalanced data, metrics such as precision, recall and F1 score are very appropriate.

3.2 RestNet50

The deeper the network, the more its performance deteriorates due to the fact that the gradient becomes very small as it progresses through the layers (that's vanishing gradient problem) and it is difficult to get good parameters for each layer. To handle these problem, ResNet has been implemented by introducing residual connexion (as shown in figure 3.4) that allow the gradient to circulate directly through the network, bypassing some layers [19]. ResNet is used for classification (it is often used as backbone for detection and segmentation) and we are going to focus on ResNet50 represented in figure 3.5.

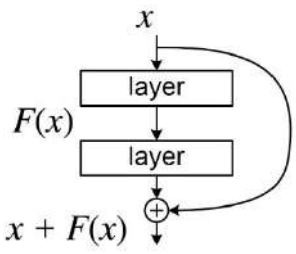


Figure 3.4: Residual connection [20]

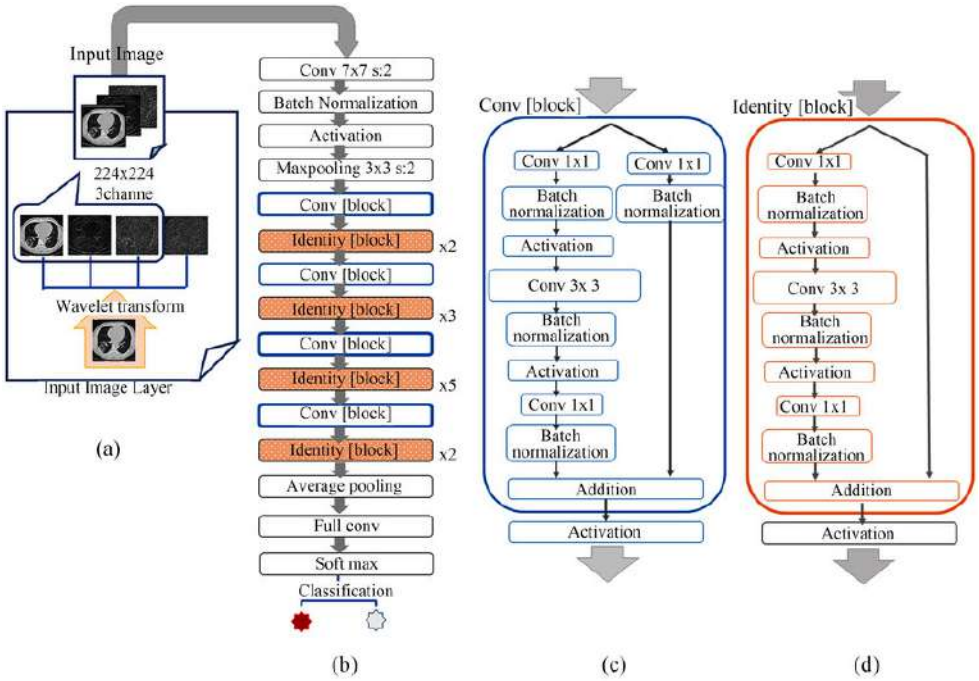


Figure 3.5: Architecture of Resnet50 [20]

Resnet-50 consists into 05 parts without counting input and output [21]:

- First layer: Here input images rise several transformations. They are passed through a convolution layer 7x7 that reduces their size by half i.e from 224x224 to 112x112. After patterns such as contours, basic textures and color gradations are extracted. Then they went into a 3x3 pooling layer further reducing their size by half i.e from 112x112 to 56x56, allowing to conserve only most important informations. The aim of this part is to capture basic information of images and simplify calculations before moving on to complex pattern extraction in deep layers.
- Set of 04 bottleneck (residual block): The first layer is followed by these bottlenecks of sub-blocks that make up respectively of 09, 12, 18 and 8 layers. Each sub-block consists of 03 layers that are respectively 1x1 conv, 3x3 conv and 1x1 conv. Each of sub-block is followed by skip connection:
  - In 1x1 conv layer 64 kernel are applied, allowing to reduce the dimension of feature maps from the preview layer.
  - 3x3 conv with 64 kernel for extracting of spacial patterns.
  - 1x1 conv with 256 kernel is applied on the output from 3x3 to restore the size of the original feature maps that is 56x56x256.
  - Finally skip connection is added to allow the stability of gradient and performance without degradation.

  In this architecture each bottleneck applied 03 ReLu including two after 1x1 conv (first one) and 3x3 conv that followed by Batch normalization , and after each skip connection. This combination of Batch + ReLu normalize the output of each layer to stabilize the rang of activations and introduce non-linearity for the best propagation of the gradient. Moreover as the network becomes deeper and deeper the number of filters is multiplied by two and the dimension of feature maps is

divided by half inside each block. That means "4 block" has 512 kernels and 28x28 as dimension of feature maps,"5 block" has 1024 kernels and 14x14 the dimension of feature maps and so on. This succession of 4 blocks with residual connections allows effective extraction of complex and simple patterns from inputs (images).

- Classification layer (Average Pooling+fully-connected): In this part, feature map from preview layer are resized using Average polling before being entered into the cf for classification.

3.3 DenseNet121

DenseNet121 (Dense Convolutional Network) is a CNN architecture which is used for classification and sometime as a backbone in several architecture for detection and segmentation [22]. It showed good performance on several image classification datasets. The objective is the same as for Resnet to solve Vanishing gradient, optimize performance and avoid the loss of information during training [22].The only difference is that ResNet uses skip connection while DenseNet uses dense connection within layers. In the dense connection each layer is connected to all previews layers inside the same block. That means the output of each layer inside the same block are concatenated and become the input of the next layer [23]. Any Densenet architecture take images as inputs, then extract feature map using dense blocks and transition layer before apply fully-connected layer to predict classes. So we will focus on DenseNet121 and present the architecture, advantages and disadvantages.

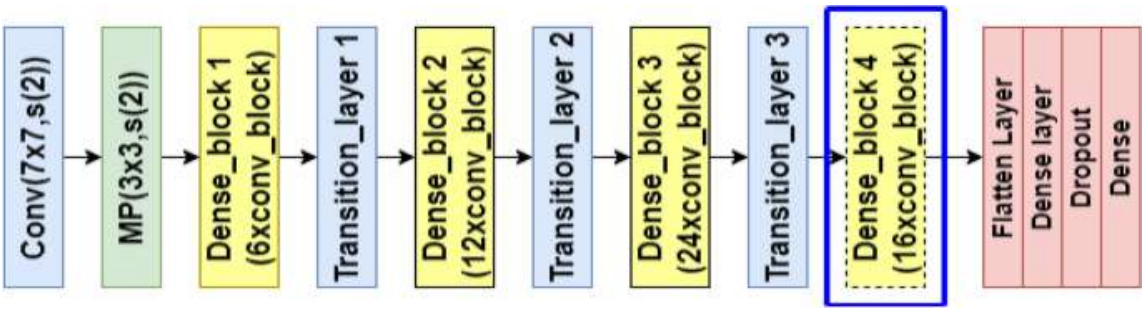


Figure 3.6: Architecture of DenseNet121 [24]

As illustrated in the figure above, which depicts the architecture of DenseNet121, we can observe that the network is composed of three main parts: input layer, 04 dense blocks separated by transition layer and classification layer [25].

- Input layer [22]:
  This layer extracts important visible information and optimizes resources. Thus, the initial stage of image processing is performed at this point, involving the application of two key operations.
  - 7x7 conv with stride=2 to reduce the dimension of images from 224x224 to 112x112 and capture the most visible elements from images such as homogeneous color zone, marked edges and others. Then feature maps are generated.
  - 3x3 max pooling to reduce the dimension of feature maps from 112x112 to 56x56.

- Dense blocks [22, 26]:
  This part consists of 04 dense blocks of 116 layers. Inside the first block, there are 6 consecutive layers (12 layers in total), where each layer follows the sequence: BatchNorm $\rightarrow$ ReLU $\rightarrow$ Conv1×1 $\rightarrow$ BatchNorm $\rightarrow$ ReLU $\rightarrow$ Conv3×3. BatchNorm + ReLu is first applied to stabilize the learning and introduce nonlinearity to filter the previously extracted feature values to only relevant information. Next, Con 1x1 is applied to reduce the number of channels by half and is followed by BatchNorm + ReLu to prepare the feature maps before being introduced into conv 3x3 for elemental pattern extraction with a 3x3 kernel and generating in turn the feature maps of size 56x56x256. Finally, this output is concatenated with all other outputs to produce a final block output of size 28x28x256. After this block, the final output undergoes transformations (BatchNorm + ReLu + Conv 1x1 + AVG pooling) in the transition layer to reduce the number of channels to 128 and the resolution to 28x28 before using them in the next dense block.

  This process is the same in all blocks, the only differences is the following: In the second block, features such as geometric shapes and complex patterns are learned with more layers (24 layers), in the third block (48 layers) advanced semantic features such as partial objects with a smaller resolution of 14x14 are learned and finally in the fourth block (32 layers) global features are summarized at a very low resolution of 7x7 by grouping all spatial and semantic information. Each block is linked to the others by a transition layer.

- Classification layer [22, 26]:
  This part is actually made up of two layers: Global Average pooling and Fully-connected + softmax. First, the output of the dense blocks is used as input to the first layer, to be reduced to a single vector of size 1x1x1024 containing the average value of each of the feature maps. This vector is then converted into a score and transformed into a probability using softmax.

Although Densenet121 offers advantages such as adaptability to small data sets and the use of a small number of parameters compared with Resnet50 thanks to its dense connections, it remains very expensive in terms of memory [26].

3.4 RexNet-150

The need to deploy CNN-based computer vision solutions on mobile and embedded devices gave rise to Rexnet. Rexnet's aim is to offer a good balance between precision and efficiency, while reducing computational complexity and maintaining competitive performance. Rexnet is one of CNN architectures that is primary used for classification tasks and as backbone in several models for detection and segmentation tasks [27]. Unlike Resnet and Densenet architectures, Rexnet uses residual inverse blocks and depthwise convolution for efficient feature extraction. These blocks make up 03 stages that are: 1x1 conv (or expansion) for increasing the number of channels, Depthwise 3x3 Conv for reducing of calculation costs, and finally 1x1 Conv for reducing the number of channels in the output of block. In addition, Rexnet uses the Swish activation function, an efficient alternative to ReLu that has shown good performance in deep networks. This function enables better gradient propagation. However we are going to focus on Rexnet-150 architecture.
Rexnet-150 consists of 351 layers, structured as follows: of an input layer, followed by a succession of 07 residual inverse blocks, each with 08 layers, and finally an output or classification layer [28].

- Input layer:
  Here, a 3x3 Conv with stride=2 is applied to reduce the dimension from 224x224 to 112x112 and increase the number of channels from8 to 16. Then, BatchNorm + Swish is applied to extracted characteristics to stabilize activation and learning. The aim is to extract complex visual features from input images and prepare them for deep blocks.

- 07 residual inverse blocks:
  As we explained above residual inverse block makes up 4 elements (1x1 conv, 3x3 conv, 1x1 conv and swish activation function). The first block aims to learn basic textures such as edges with output characteristics of size 112x112x24. The second block performs this sequence of operations twice to reduce the resolution of feature maps and learn patterns such as geometrical shapes (with output 56x56x40). Block after block, complex patterns are progressively learned until the 07th block, when the final extraction is carried out on a small 7x7x320 resolution with more abstract, complex and relevant information.

- Output layer:
  Global Average pooling is applied to the output of residual blocks to transform this output into a 1D vector, making each channel available at all spatial positions. The aim is to summarize the spatial information while retaining the channel characteristics. This vector is then passed to the fully connected layer for prediction, associating the characteristics with a class. Finally Softmax is applied to produce class probabilities.

This architecture offers a number of advantages over Resnet-50 and Densenet-121, such as a gradual increase in the number of channels rather than a reduction in the number of channels in blocks, as is the case with the other architectures studied, improved gradient propagation thanks to the Swish activation function, and better performance with a small number of parameters. However, although it has these advantages, it also has limitations, such as the high number of layers and the fact that it is less widely used and studied.

4. Implementation

4.1 Data annotation and labeling

The dataset comprises 4,507 images of African plums, captured under natural lighting conditions between 8:00 AM and 4:00 PM, from multiple angles, and featuring plums of varying sizes. The data were collected across six regions of Cameroon Centre, East, Littoral, South-West, West, and Adamawa over a two-year period (2021-2023) [29], in order to capture the natural biodiversity and environmental variability. Each image has a resolution of 640x640 pixels and is labeled into one of six quality categories based on the visible state of the fruit: bruised, rotten, cracked, spotted, unripe, and unaffected, as illustrated in



Figure 4.1: Sample of a plum in each class

According to [29], each that is defined by:

- Bruised: These are characterized by damaged areas of brownish or purplish color, often soft, caused by shocks during rough transport, harvesting or improper storage.
- Cracked: These plums are marked by the presence of cracks or crazing, generally caused by environmental factors such as excess moisture.
- Rotten: These are visibly deteriorated plums characterized by soft skin, liquefied flesh with brownish-black, greenish spots, a fermenting odor and areas of decomposition.
- Spotted: These are marked by the presence of visible imperfections on their skin caused by insect bites, fungal infections, physiological defects a such as sunburn and mineral deficiencies, or rubbing against branches.
- Unripe: These are plums that are not directly edible and have been harvested before they are ripe. They have an acid, bitter or bland taste and a hard texture.

- Unaffected: This is the category of plums with no visible defects, i.e. the good ones.

Each of the classes forms a sub-folder named after the class, and all are in a main folder. These classes are distributed as shown figure 4.2:

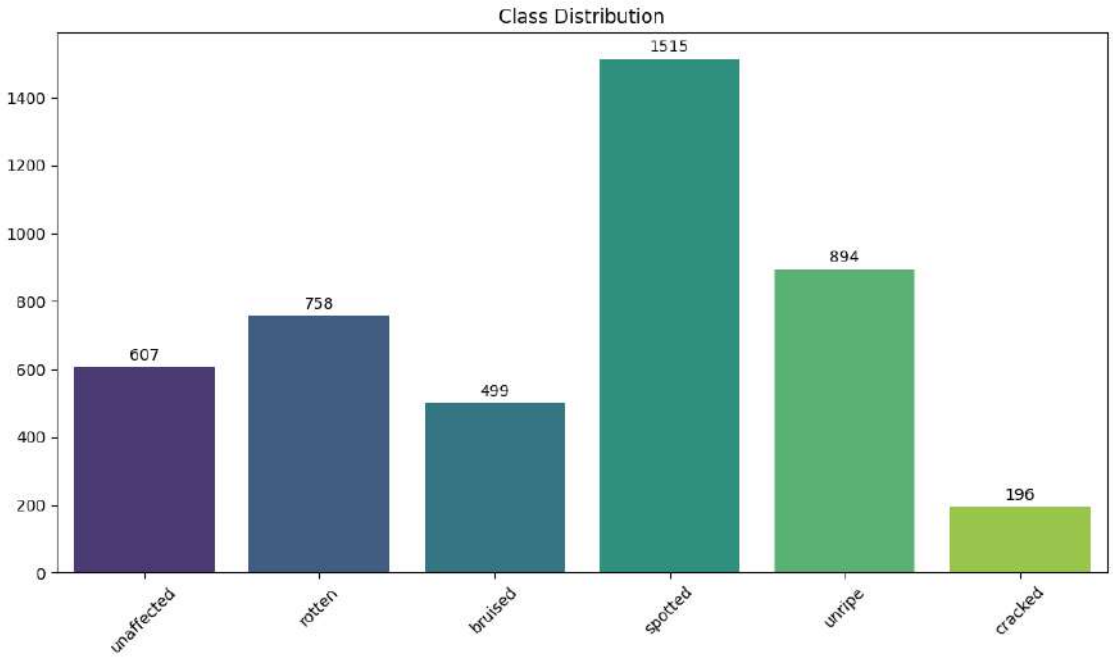


Figure 4.2: Distribution of classes

The spotted class is the most represented, comprising 1,515 images, followed by the unripe class with 894 images. In contrast, the cracked and bruised classes are underrepresented, with only 196 and 499 images respectively.

4.2 Data processing

4.2.1 Preprocessing. This aspect is crucial for effective modeling, as a well-prepared and properly preprocessed dataset significantly contributes to the performance of machine learning models. Moreover, it is essential to select methods that are well-suited to the quality and nature of the data. Accordingly, in this research, the following methods were applied:

- Verification of annotation: This is to ensure that every images are in the right class. During this stage some images (3,011 images) have been identified and reclassified according to their above-mentioned characteristics.
- Resizing, normalization and balancing: Knowing that the different classes are imbalanced, it was mandatory to balance them by using appropriated method to balance. In our case, we used WeigthRandomSampler method which assigns high weights to under-represented classes and lower weights to others. Assigning high weights to the less represented classes allows the model to select the sample from these classes more often, with a view to seeing as many images of each

class. This method forces the model to learn rare classes during training. This method is defined mathematically:

$$\text{Weight}_{\text{class}_i} = \frac{\text{Total number of samples}}{\text{Number of classes} \times \text{Number of samples in class}_i} \tag{4.2.1}$$

Then, these images are resized to 224x224 and finally normalization is applied to each pixel of images by channel (we have 3 in our case) after converting images in tensor following the next formula:

$$\text{Nornmalized pixel} = \frac{\text{Pixel of image} - \text{mean}}{\text{std}} \tag{4.2.2}$$

Where mean and std are the mean and standard deviation of the channel which are computed on Imagenet's dataset (This is a set of 14 million object images used in computer vision and machine learning).

4.2.2 Data Splitting. After preparing and preprocessing our data, we divided it into train, validation and test set into 90%, 5% and 5% respectively. This is done using the stratified method to ensure consistent representation of the classes in the different sets.

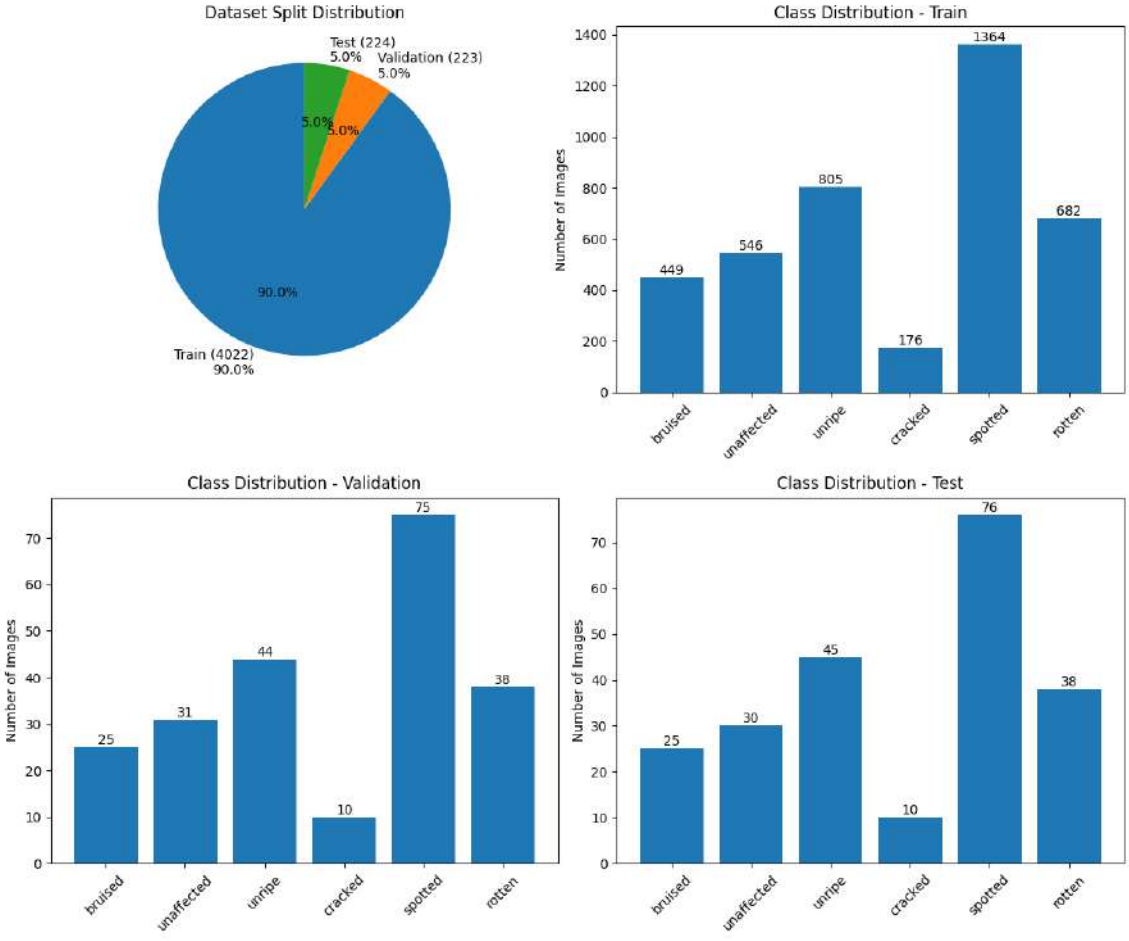


Figure 4.3: Distribution of sets

As shown figure 4.3, we have 4022 images in train set, 224 in test set and 223 in validation set. This shows also the same distribution inside the different sets.

4.3 Model training and validation

4.3.1 Model Selection. The dataset contains images that vary in quality from blurred to sharp with clear details. The objects contained (Safou) in these images also vary in texture, color (light pink, violet and dark blue) and even shape. Given these variations, CNN models are more appropriate for this classification task, notably DenseNet-121, ResNet-50 and RexNet-150, which have shown different performances in terms of efficiency and precision in several computer vision tasks, thanks to their ability to learn complex characteristics.

To achieve this, we have used the python programming language and implemented it on VS code and Google Colab. All necessary libraries are installed before loading the dataset to apply the above methods.

4.3.2 Training Process. We created a .ipynb file for each model and each model has its parameters:

- DenseNet-121: To train the model, we began by initializing the pre-trained network and freezing the last 20 layers to enable the learning of features specific to Dacryodes edulis (Safou). The final classification layer was replaced with a custom architecture comprising a global pooling layer, a dense layer with 256 neurons, and a fully connected output layer with 6 neurons activated by a softmax function corresponding to the six target classes. To improve generalization and prevent overfitting, we incorporated three dropout layers and implemented early stopping. The model was trained for 10 epochs using images resized to 224x224 pixels, a batch size of 32, a learning rate of 1e-4, and the Adam optimizer. The training process was computationally intensive, requiring approximately 17 hours to complete.
- RexNet-150: Training began with model initialization, during which all layers were frozen except for the final fully connected layer. This layer was replaced with a new fully connected layer consisting of 6 neurons, allowing the model to adapt to the specific classification task. The model was trained using the Adam optimizer with a learning rate of 3e-4, a batch size of 32, and input images resized to 224x224 pixels. The training process spanned approximately 23 hours
- ResNet-50. The process is the same as for RexNet-150, and the same hyperparameters are used. This trainning took 22 hours.

4.3.3 Validation and Evaluation. For the validation of models, we used val set and at each epoch, the model is evaluated on vvalidation set using metrics : loss function, F1 score, precision and accuracy. But for the final evaluation we use more metrics such as Recall and confusion matrix.

5. Results and discussion

As we mentioned above, we used F1 score, precision, accuracy, loss function and matrix as metrics to get the following results for each model:

5.1 DenseNet-121

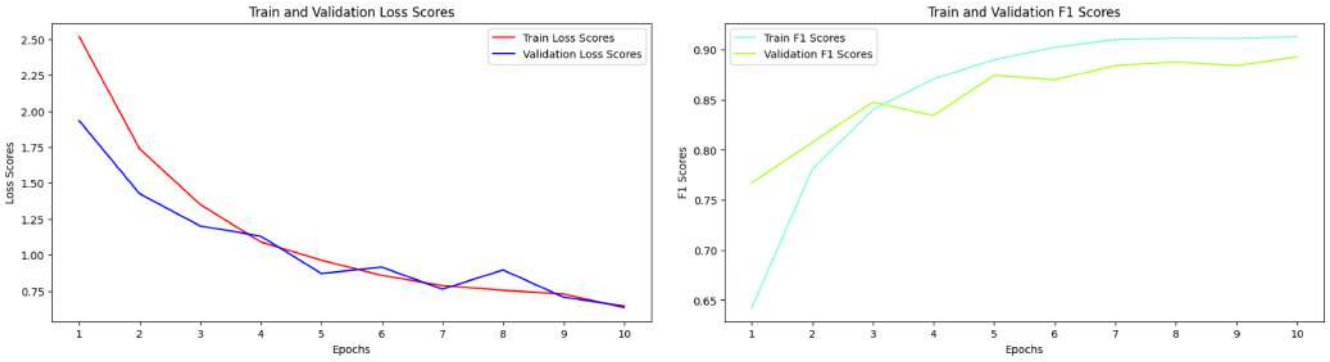


Figure 5.1: Train and validation loss, train and validation F1 score

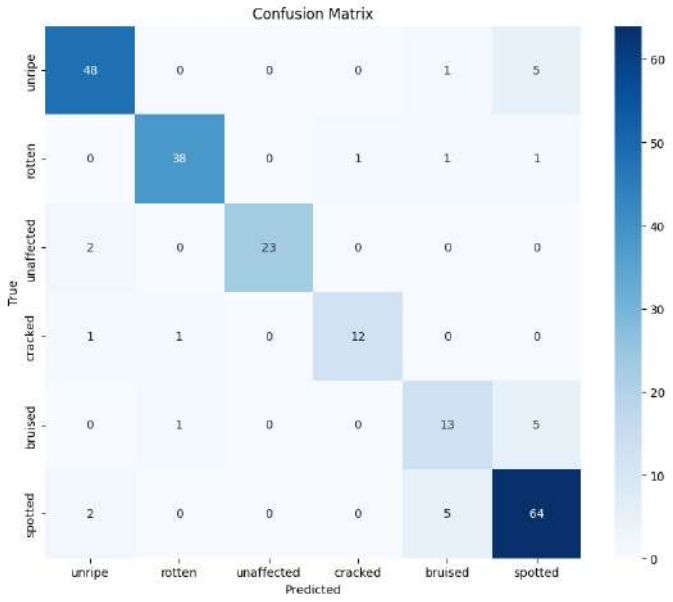


Figure 5.2: Confusion matrix

Figure 5.1 shows on the left the loss function of the train and validation on all epochs, and on the right their F1 score.The train and validation F1 scores increase together with very close values from epoch 2 to epoch 10. However, from epoch 2 onwards, the train's F1 score remains slightly above that of the validation, with an almost constant difference of around 2%. In addition, the train loss function gradually decreases from 2.52 to 0.63, where it begins to oscillate around 91.3%. By contrast, the

validation loss function shows alternating increasing and decay over the epochs (from the fifth to the eigth epoch). However, its overall pattern remains decreasing, but less regular than that of the train. In addition, from the eigth epoch onwards, the two loss functions converge and become totally close.

The model was later on evaluated on a test set. The result is shown in figure 5.2 representing the classification report highlighting the model's strengths and weaknesses. It shows that the unaffected, rotten and spotted classes seem to be better learned than the others.

- The unripe class has well-predicted 48 plums and poorly predicted six (6).
- The bruised class has 13 well-predicted plums, and 6 misclassified plums.
- Similarly, out of 41 plums, 38 are well predicted, compared to three (3) in the rotten class.
- Out of the 12 plums in the cracked class, two (2) were misclassified
- Two (2) out of 25 plums were misclassified in the unaffected class.
- Finally, in the spotted class, seven (7) plums were misclassified out of 71 plums.

The classification report shows that the model is able to better recognize the characteristics of rotten, spotted and unaffected classes than the others. In addition, the model has a performance of 88.4% F1 score and 88.4% recall on the test set. This means that the model is able to classify a plum at 88.4%.



Figure 5.3: Predicted plums by DenseNet-121

Figure 5.3 represents predicted plums. We note that two spotted and rotten plums were predicted as bruised plums. These two plums have all the characteristics of the bruised class, which explains the misclassification mentioned above.

5.2 RexNet-150

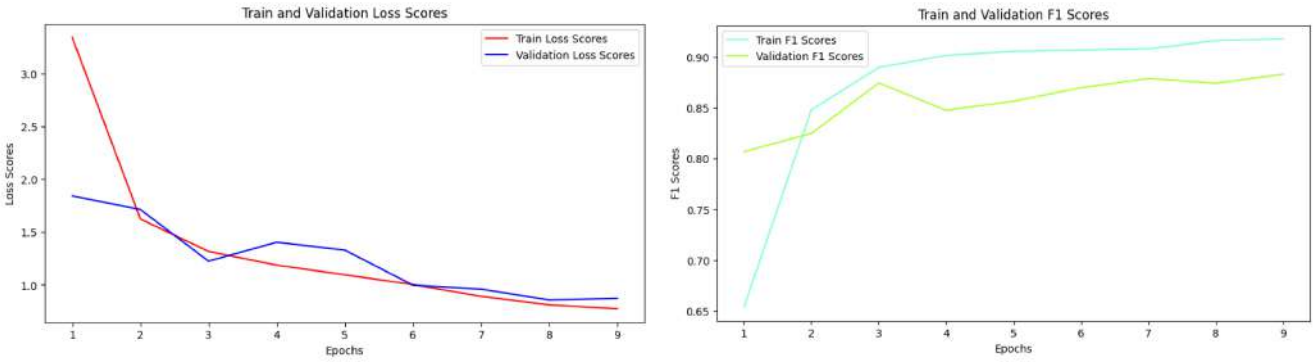


Figure 5.4: Train and validation loss, train and validation F1 score of RexNet-150

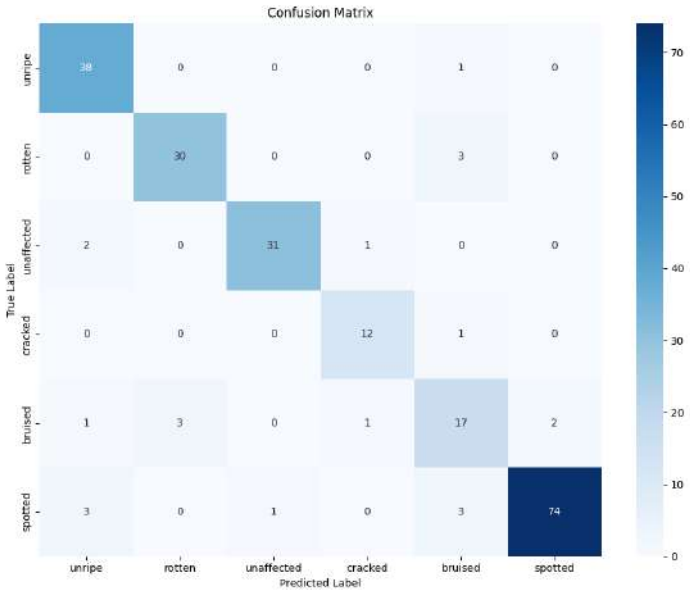


Figure 5.5: Confusion matrix of RexNet-150

Figure 5.4 shows an alternating rise and fall in the loss val, from 1.843 to 0.873. However, the loss train shows a good, gradual and steady decline from 3.342 to 0.77, unlike the loss val. Overall, these two losses are decreasing and the gap between them remains minimal. This shows good convergence. The F1 score train grows rapidly from epoch 1 to 3, then continues to grow progressively. In contrast, F1 score validation shows a succession of growth and decay between epochs 1 and 4, then continues to grow progressively and steadily. However, from epoch 3 to the end we observe a reasonable 3.5% difference between the two F1 scores, which seems constant overall. These reach 91.8% (F1 score train) and 88.3% (F1 score validation).

From these graphs, we can say that the model has a good capacity to learn from the characteristics of different classes and to generalize well.

As shown in figure 5.5 and table 5.1, the model performs well in all classes except bruised.

Table 5.1: Classification Report

Class	Precision	Recall	F1-Score
unripe	0.86	0.97	0.92
rotten	0.91	0.91	0.91
unaffected	0.97	0.91	0.94
cracked	0.86	0.92	0.89
bruised	0.68	0.71	0.69
spotted	0.97	0.91	0.94

- In the unripe class, 38 plums out of 39 are very well classified, demonstrating the model's ability to recognize its characteristics at 97%.
- The model performs well at 91% for the rotten (30 out of 33 are well classified), unaffected (31 out of 34), spotted (74 out of 81) and cracked (12 out of 13) classes.
- For the bruised class, on the other hand, performance is 71%.

Although the model shows an overall excellent performance of 90% accuracy, it is likely to confuse the characteristics of a given class with another. This is notably the case for the unripe, rotten, cracked and spotted classes, which are confused to bruised class.

5.3 Resnet-50

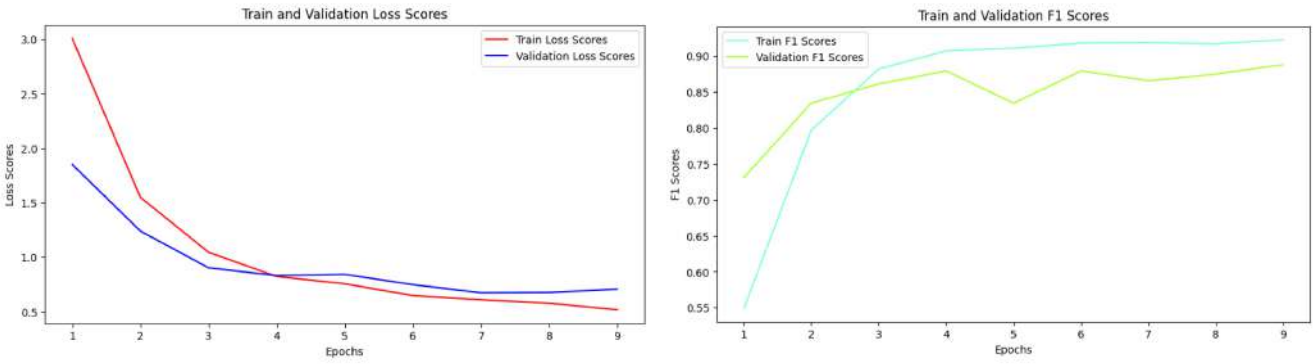


Figure 5.6: Loss train & validation, and F1 score train & validation of ResNet-50

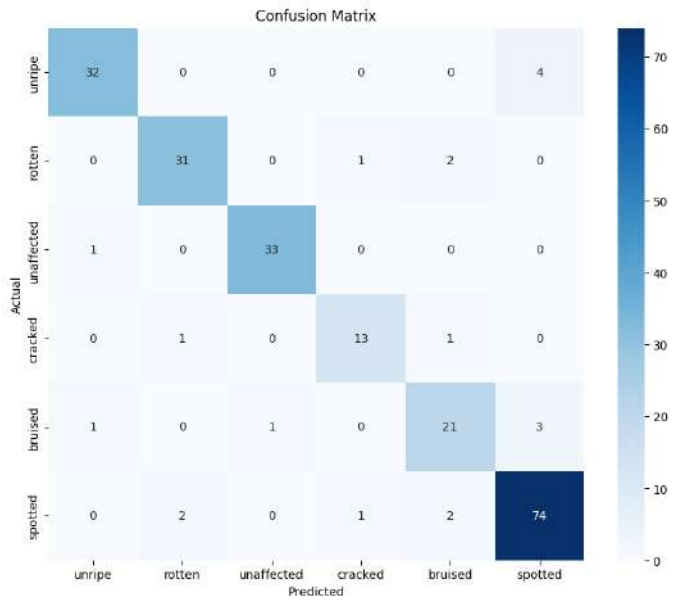


Figure 5.7: Confusion matrix of ResNet-50

The train loss function is superior to the validation function from the first epoch to the fourth epoch, when it becomes inferior to the validation function, but they remain very close. Overall, train and validation losses decrease progressively until epoch 7, when they begin to converge slightly as shown in figure 5.6. Concerning F1 score, the train F1 score is very low at the start (0.55) compared to the validation score (0.738). Then the train F1 score increases very quickly until it becomes higher than the validation F1 score after epoch 2 until epoch 9 where it reaches a score of 0.922. However, overall, both F1 scores increase progressively and slightly from epoch 4 on wards. But this difference between the two remains constant and is around 0.32. What's more, the model is evaluated on the test set after training and has achieved an F1 score of 90.0%, with precision of 90.10% and accuracy of 91.07%. This shows how well the model generalizes on data it has never seen before.

Table 5.2: Classification Report

Class	**Precision**	**Recall**	**F1-Score**
unripe	0.9412	0.8889	0.9143
rotten	0.9118	0.9118	0.9118
unaffected	0.9706	0.9706	0.9706
cracked	0.8667	0.8667	0.8667
bruised	0.8077	0.8077	0.8077
spotted	0.9136	0.9367	0.9250

In addition, the classification ratio illustrated by figure 5.7 and table 5.2 of this model looks good. The unaffected, spotted and rotten classes show excellent ratios, with very few instances mispredicted (1 out of 34, 5 out of 79 and 3 out of 34 respectively). Their rates of well-classified plums are 97.06%, 93.67% and 91.18% respectively as shown in table 5.2. However, the unripe, cracked and bruised classes also show good ratios, but less than the latter, with 88.88%, 86.66% and 80.78% respectively of well-predicted plum rates. However, some plums in a given class are confused with others, but this error is more marked in the unripe, bruised and cracked classes. In the unripe class, 4 instances are predicted

as spotted, in bruised 5 as unripe 1 as unaffected and 3 as spotted. Finally, 2 plums are confused as bruised and rotten in the cracked class.

Thus, the model is able to distinguish the different classes at 91.07% and these errors are due to the visual similarity between these different classes, namely the fact that one plum has the visual characteristics of one or more other classes. As shown in figure 5.8, the first plum has the characteristics of an unripe and spotted plum. Similarly, the plum on the right also displays the characteristics of two classes, cracked and bruised.



Figure 5.8: Error

After training and evaluation of the model, it 's important to visualize the predicted images as shown in figure 5.9 (where 0: unaffected, 1: rotten, 2: bruised, 3: unripe, 4: spotted and 5: cracked).

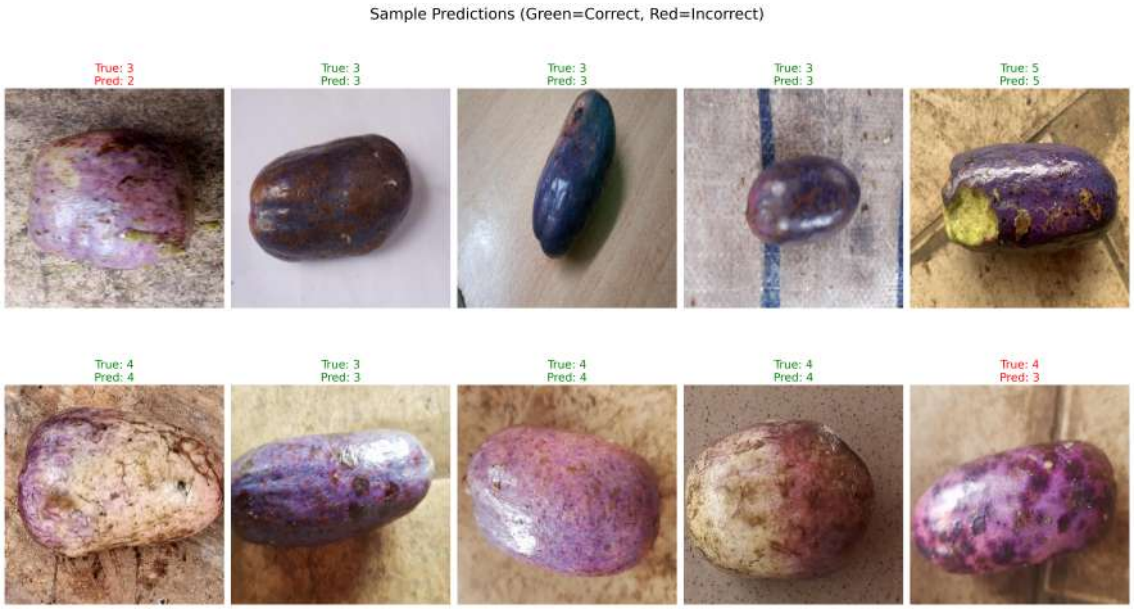


Figure 5.9: Predicted instances by Resnet-50

Here, the model misclassified unripe plum as bruised and, spotted plum as unripe.

In addition, we have the summary of the performance of these models in table 5.3.

Table 5.3: Summary of Model Performance Metrics

Model	F1 Score (%)	Precision (%)	Recall (%)	Accuracy (%)
DenseNet-121	88.4	88.4	88.4	88.4
RexNet-150	88.3	86.6	90.02	90.0
ResNet-50	90.0	90.19	90.0	91.07

Overall, the RexNet-150 and ResNet-50 models perform very similarly, with a general accuracy of 90% and 91.07% respectively (as shown on table 5.3). DenseNet-121, on the other hand, deviates completely, with an overall accuracy of 88.4%. This makes ResNet-50 the best model, with a very good ability to distinguish the different classes. These models could be further optimized, however they are limited by:

- Image quality: in some images, it's difficult to distinguish the background from the object, and others are noisy.
- The visual similarity of plums: some plums, in addition to their own class characteristics, also display those of other classes.
- The imbalance of classes, notably cracked, bruised and unaffected, which are grossly under-represented.
- Computational resources.

Conclusion and future work

The aim of this research was to implement deep learning models (DenseNet-121, ResNet-50 and RexNet-150) based on CNN architecture for multi-class classification of African plums according to whether they are bruised, cracked, spotted, unripe, rotten or unaffected. These models were trained on a dataset of 4507 images. In addition, pre-processing methods such as normalization and class balancing via weigthRandomSampler were applied to the data, then the different models trained before being evaluated. The results obtained show that the ResNet-50 model outperformed the others with an accuracy of 91.07%.

To extend the impact of this research, future work will focus on two main directions. First, the most efficient deep learning model ResNet-50 will be deployed in a user-friendly mobile or web application to facilitate real-world usage by farmers as an initial prototype for automated plum sorting. This will allow for practical testing and iterative improvement based on user feedback in agricultural settings. Second, the classification framework will be expanded to incorporate not only visual features but also chemical or biochemical characteristics of the fruit. By integrating spectroscopic or sensor-based data, it will be possible to assess both the external appearance and internal quality of the plums. This dual approach aims to ensure that plums reaching the market are not only visually appealing but also meet standards of taste and nutritional value.

Acknowledgements

This work could never have been carried out without the support and contribution of certain people to whom I express my gratitude. Thus, I wish to thank the Lord for the health and grace granted.

Through this work, I would like to honor my family for their unconditional support and encouragement throughout this study, especially my dear parents, Kochiake Bossera and Koubobe Rebecca.

I would also like to sincerely thank my supervisor Dr. Arnaud Fadja Nguembang and my tutor Mme Idene Mantho for their availability, guidance and contribution to the success of this work.

I would also like to thank Prof. Mama Foupouagnigni, the center president of AIMS Cameroon and Dr. Daniel Tcheutia, the academic director, for their efforts in my training; the IT team for their assistance. I can't conclude without thanking the entire 2024-2025 Batch of students, especially Lydia Tiapo, Djasrabe Emmanuel, Mohammed Pagna, Oumar Moussa, Ishmael Ayen, Kenfack Annie and Leslie Djoumessap.

References

[1] K. Schreckenberg, A. Degrande, C. Mbosso, Z. Boli Baboule, C. Boyd, L. Enyong, J. Kanmegne, and C. Ngong. The social and economic importance of dacryodes edulis (g.don) h.j. lam in southern cameroon. *Forests, Trees and Livelihoods*, 12(1-2):15–40, 2002.

[2] Nsioscar. African pear (eben) in Ibibio culture in Nigeria. https://hive.blog/hive-120586/@nsioscar/african-pear-eben-in-ibibio-culture-in-nigeria, 2023. Accessed May 15, 2025.

[3] Abdon Awono, Ousseynou Ndoye, Kathrin Schreckenberg, Honore Tabuna, Fernand Isseri, and Ludovic Temple. Production and marketing of safou (dacryodes edulis) in cameroon and internationally: market development issues. *Forests, Trees and Livelihoods*, 12(1-2):125–147, 2002.

[4] Mengyang Wu, Ya Qiu, Wenying Wang, Xun Su, Yuhao Cao, and Yun Bai. Improved rt-detr and its application to fruit ripeness detection. *Frontiers in Plant Science*, 16:1423682, 2025.

[5] Inkyu Sa, Zongyuan Ge, Feras Dayoub, Ben Upcroft, Tristan Perez, and Chris McCool. Deepfruits: A fruit detection system using deep neural networks. *Sensors*, 16(8):1222, 2016.

[6] Weikuan Jia et al. Detection and segmentation of overlapped fruits based on optimized mask r-cnn application in apple harvesting robot. *Computers and Electronics in Agriculture*, 172:105380, 2020.

[7] Kavita Goyal, Pardeep Kumar, and Kavita Verma. Ai-based fruit identification and quality detection system. *Multimedia Tools and Applications*, 82(16):24573–24604, 2023.

[8] A. N. Fadja, S. R. Che, and M. Atemkemg. Intelligent vision system with pruning and web interface for real-time defect detection on african plum surfaces. *Information*, 15(10):635, 2024.

[9] Dulari Bhatt, Chirag Patel, Hardik Talsania, Jigar Patel, Rasmika Vaghela, Sharnil Pandya, Kirit Modi, and Hemant Ghayvat. Cnn variants for computer vision: History, architecture, application, challenges and future scope. *Electronics*, 10(20):2470, 2021.

[10] D. Dai. An introduction of cnn: Models and training on neural network models. In *2021 International Conference on Big Data, Artificial Intelligence and Risk Management (ICBAR)*, pages 135–138. IEEE, 2021.

[11] Yash Shah, Parth Shah, Mansi Patel, Chinmay Khamkar, and Pratik Kanani. Deep learning model-based multimedia forgery detection. In *Proceedings of the Fourth International Conference on I-SMAC (IoT in Social, Mobile, Analytics and Cloud) (I-SMAC)*, pages 564–572. IEEE, 2020.

[12] Joshua Ayobami Ayeni. Convolutional neural network (cnn): The architecture and applications. *Applied Journal of Physical Science*, pages 44–50, December 2022.

[13] Vitalflux. Different types of cnn architectures explained with examples, 2023. Accessed: 2023-10-05.

[14] Serdar Abut, Hayrettin Okut, and K. James Kallail. Paradigm shift from artificial neural networks (anns) to deep convolutional neural networks (dcnns) in the field of medical image processing. *Expert Systems with Applications*, 244:122983, 2024.

[15] Stanford University. Cs231n: Convolutional neural networks for visual recognition. https://cs231n.github.io/convolutional-networks/, 2023. Accessed: 2023-10-15.

[16] GeeksforGeeks. Activation functions in neural networks. https://www.geeksforgeeks.org/activation-functions-neural-networks/, 2023. Accessed: 2023-09-30.

[17] Yunji Chen, Ling Li, Wei Li, Qi Guo, Zidong Du, and Zichen Xu. Chapter 2 - fundamentals of neural networks. In Yunji Chen, Ling Li, Wei Li, Qi Guo, Zidong Du, and Zichen Xu, editors, *AI Computing Systems*, pages 17–51. Morgan Kaufmann, 2024.

[18] Hidenori Ide and Takio Kurita. Improvement of learning for cnn with relu activation by sparse regularization. In *Proceedings of the International Joint Conference on Neural Networks (IJCNN)*, pages 1966–1985, 2017.

[19] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778. IEEE, 2016.

[20] Eri Matsuyama. A deep learning interpretable model for novel coronavirus disease (covid-19) screening with chest ct images. *Journal of Biomedical Science and Engineering*, 13(7):140–152, 2020.

[21] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. *arXiv preprint arXiv:1512.03385*, 2015.

[22] Busireddy Seshakagari Haranadha Reddy, R Venkatramana, and L Jayasree. Enhancing apple fruit quality detection with augmented yolov3 deep learning algorithm. *International Journal of Human Computations & Intelligence*, 4(1):385–396, 2025.

[23] Aswin S Vellaichamy, Akshay Swaminathan, C Varun, and S Kalaivani. Multiple plant leaf disease classification using densenet-121 architecture. *International Journal of Electrical Engineering and Technology (IJEET)*, 12(5):38–57, 2021.

[24] Sudeep D. Thepade and Sandeep Chauhan. Enhancing land use identification through fusion of densenet121 dcnn and thepade sbtc features using machine learning algorithms and ensembles. *SN Computer Science*, 4:772, 2023.

[25] Y Huang, L Lin, P Cheng, J Lyu, R Tam, and X Tang. Identifying the key components in resnet-50 for diabetic retinopathy grading from fundus images: A systematic investigation. *Diagnostics*, 13(10), 2023.

[26] Pandi Barita Nauli Simangunsong, Poltak Sihombing, Syahril Efendi, and Fahmi Fahmi. Densenet-121 analysis for classification of tomato leaf diseases. In *2024 7th International Seminar on Research of Information Technology and Intelligent Systems (ISRITI)*, pages 907–911, 2024.

[27] Dongyoon Han, Sangdoo Yun, Byeongho Heo, and YoungJoon Yoo. Rethinking channel dimensions for efficient model design. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 732–741. IEEE, 2021.

[28] Xuwei Qian, Renlong Hang, and Qingshan Liu. Rex: An efficient approach to reducing memory cost in image classification. *arXiv preprint arXiv:2106.20106*, 2021.

[29] Arnaud Nguembang Fadja, Armel Gabin Fameni Tagni, Sain Rigobert Che, and Marcellin Atemkeng. A dataset of annotated african plum images from cameroon for ai-based quality assessment. *Data in Brief*, 59:111351, 2025.